

Peer-to-Peer KV-Cache Sharing for Disaggregated LLM Serving

Nili Guy

Liran Schour

(author list TBD)

IBM Research

Draft — July 19, 2026

Abstract

Prefix KV caches in distributed LLM serving are per-instance resources: a prefix cached on one pod is recomputed on every other pod that serves it. We describe a peer-to-peer (P2P) KV-cache sharing mechanism for the llm-d inference stack in which any vLLM instance can pull cached prefix blocks from a peer’s CPU offload tier over the network, under a router-side policy that requests a pull only when the expected saving exceeds a calibrated threshold. The mechanism generalizes the prefill/decode (P/D) disaggregation transfer path into a symmetric peer protocol and composes with prefix-affinity routing, KV offloading, and P/D disaggregation. We evaluate on three models and four workload families on H200 clusters. A single-request analysis shows the pull outperforming recomputation above a model-dependent crossover of 300–900 tokens, with the advantage growing with prefix length (−68% latency at 48K tokens). End to end, adding P2P to the llm-d P/D reference deployment is strictly beneficial at high load: on a document-Q&A workload, median TTFT drops 10.3× with +40% throughput; on agentic session shapes (10–100K-token contexts, tool-call idle gaps), median TTFT drops 4.8× with +33% throughput, while tail latency is bounded in both arms by the irreducible cold prefill. We further show that placement policy, not transfer capability, determines whether sharing pays: making the pull the primary data path (load-only placement) scatters cache mass and loses to prefix affinity at every load tested, whereas affinity placement with pull-on-divergence wins consistently.

1 Introduction

Reuse of prefix KV state is one of the highest-leverage optimizations in LLM serving: shared system prompts, common documents, multi-turn histories, and agentic loops all re-present token prefixes whose attention state has already been computed somewhere in the fleet. Modern routers exploit this locally — prefix-affinity routing steers a request to the pod most likely to hold its prefix — but affinity has structural limits: sessions migrate un-

der load balancing, replicas come up cold on scale-out, and under contention the pod that owns a popular prefix becomes the queueing bottleneck.

This paper describes and evaluates a complementary mechanism in the llm-d stack: *peer-to-peer KV-cache sharing*, in which the missing prefix is transferred from the peer that holds it instead of recomputed. Three design decisions distinguish the system: (i) transfers are served from the CPU offload tier, off the GPU and off the request’s failure path (a failed or partial pull degrades to recomputation); (ii) the pull decision is made centrally by the router’s endpoint picker (EPP), reusing the same per-pod prefix-cache index that powers affinity routing; and (iii) the decision is gated by a per-model calibrated threshold so a pull is requested only when the peer’s cached advantage exceeds the transfer’s break-even point.

We report a measurement-driven evaluation: a single-request pull-versus-recompute analysis that prices the mechanism per model (§3); paired end-to-end comparisons on the llm-d P/D reference topology under document-Q&A and agentic workloads (§4); a mechanism study showing router-decided prefill←decode transfers sustaining flat per-turn latency in multi-turn chat (§5); and a negative result (§6): using the pull as the primary placement strategy loses to prefix affinity at every load we tested, on two independent implementations.

2 Background and Design

Setting. llm-d deploys vLLM instances behind an inference gateway. An endpoint-picker (EPP) scores candidate pods per request; prefix-cache scorers use either an approximate index (routing history) or a precise index fed by engine KV events. vLLM’s offloading connector spills KV blocks from GPU HBM to a CPU memory tier; P/D disaggregation runs separate prefill and decode workers connected by NIXL.

Mechanism. The P2P connector generalizes the P/D transfer path into a symmetric peer protocol. Every instance can act as *producer* (serving blocks from its CPU tier) and *consumer* (pulling blocks it lacks). A consumer

Table 1: Measured pull-versus-recompute crossover and the deployed pull threshold.

model	crossover (tokens)	threshold
gpt-oss-120b	< 2 048	2 048
Llama-3.1-8B	$\approx$ 2 000	2 048
Qwen3-30B-A3B	$\approx$ 760	1 024

sends the block hashes it needs over a ZMQ control channel; the producer answers with its hits; NIXL moves the matched blocks CPU-tier to CPU-tier; hits then load into the consumer’s GPU as ordinary cache hits and misses are recomputed. Transfers are asynchronous and best-effort.

Policy. The EPP compares, for each request, the best-cached candidate against the pod that will compute the prefix (under P/D, the prefill worker). When the peer’s cached-token advantage exceeds `minCachedTokenDelta`, the EPP attaches a source header; the routing sidecar translates it into connector transfer parameters on the prefill leg. Ties and self-matches never trigger a pull.

Deployment invariants. Block hashes must agree fleet-wide (identical `--block-size` and hash seed), peers must run the same tensor-parallel degree (KV block identity is TP-dependent), and the CPU tier should be provisioned at roughly $2\times$ the *measured* GPU KV capacity, since its value is the KV that the GPU evicts and the CPU retains.

3 Single-Request Economics

Figure 1 prices the mechanism on gpt-oss-120b (one H200 per pod): prefill latency for a fully cached prefix, recomputed locally versus pulled from a peer’s CPU tier (5-repetition medians). The pull wins at every measured length and the gap widens with size, reaching -68% at 48K tokens (551 ms versus 1,695 ms). On Llama-3.1-8B the curves cross near 2K tokens; on gpt-oss the pull already wins at 2K because its KV footprint per token is small relative to its prefill speed. A two-point field procedure (time one warm pull and one fresh recompute at a small and a large size) recovers the fixed transfer overhead (≈ 30 ms) and both per-token rates, and hence the crossover, in minutes on a live pod pair; on Qwen3-30B-A3B it yields ≈ 760 tokens, and the deployments below set the threshold one power of two above their measured crossover (Table 1).

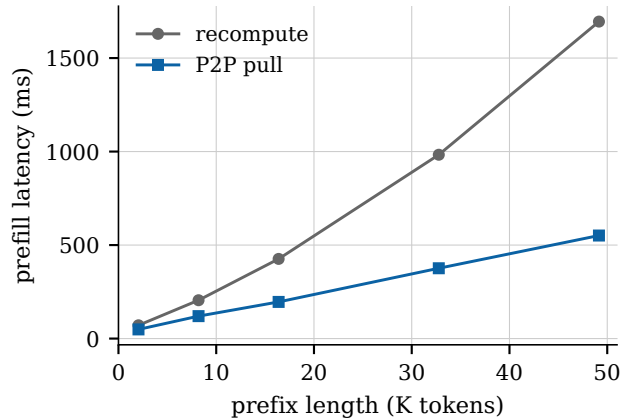


Figure 1: Prefill latency for a fully cached prefix: local recompute versus P2P pull from a peer CPU tier. gpt-oss-120b, single source-consumer pair, 5-rep medians.

4 End-to-End Results on the P/D Reference Topology

We compare the llm-d P/D reference deployment exactly as shipped (arm A) against the identical deployment plus the P2P stack — CPU offload tier and pull — with placement policy unchanged (arm B). Each arm runs on a freshly provisioned fleet; workloads and arms complete all requests with zero failures.

Document Q&A. gpt-oss-120b on $16\times$ H200 (8 prefill + 8 decode, TP=1); 192 conversations $\times$ 6 turns over ≈ 49 K-token documents at concurrency 192 (Table 2, left; Figure 2). Median TTFT improves $10.3\times$ and throughput $+40\%$. The dominant effect at this operating point is the offload tier serving history re-prefills (52M externally served tokens) that arm A recomputes under 192-deep queues.

Agentic sessions. Qwen3-30B-A3B-Thinking on $6\times$ H200 (2 prefill + 4 decode, TP=1), the agentic-serving reference shapes: 10–100K-token contexts, 4–40 turns, tool-call gaps of 1–20s that evict session KV between turns; 288 requests at concurrency 16 (Table 2, right). Median TTFT improves $4.8\times$ with $+33\%$ throughput, and 1.23M tokens of session history are pulled instead of recomputed in a 229s run (a second arm-B sample reproduces the result: p50 1.06s, 237s). The p99 is unchanged by construction: both arms’ worst case is the cold first prefill of a 100K-token context; the pull removes re-computation, not first computation.

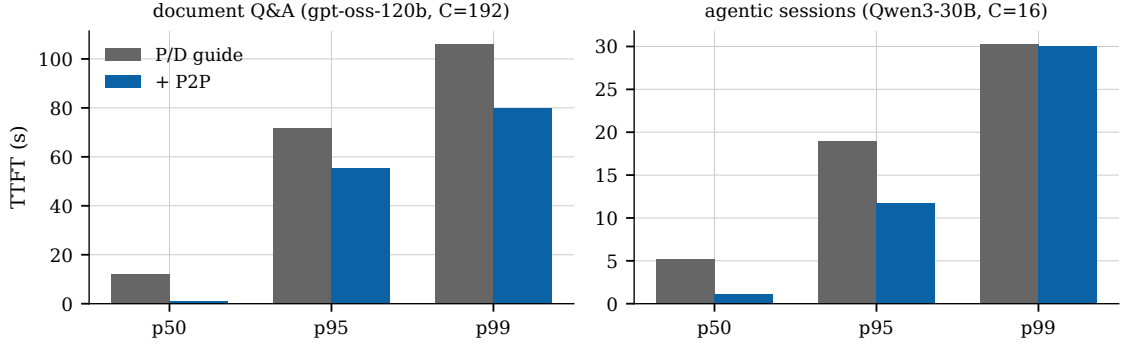


Figure 2: TTFT percentiles for the paired arms of Table 2.

Table 2: Paired end-to-end results on the P/D reference topology. All arms complete all requests (1,152 and 288 respectively) with zero failures.

metric	document Q&A		agentic	
	guide	+P2P	guide	+P2P
TTFT p50 (s)	11.94	1.16	5.22	1.09
TTFT p95 (s)	71.6	55.2	18.94	11.77
TTFT p99 (s)	106.1	80.0	30.29	29.98
throughput	5.68/s	7.96/s	288/304s	288/229s

5 Mechanism: Router-Decided Prefill←Decode Pulls

Under P/D disaggregation the newest KV of a conversation lives on the decode worker: it received the prompt over NIXL and generated the answer. On the next turn, the scheduled prefill worker lacks that history; the EPP’s precise index — fed by KV events from both roles — observes the decoder’s advantage and requests the pull. We measure this on Llama-3.1-8B chat multi-turn (4 prefill + 4 decode, 48 conversations × 8 turns, live message history): 477K tokens of session history move decoder→prefiller in one run, and per-turn TTFT holds at 0.1–0.2s while prompts grow from 5K to 20K tokens (Figure 3). On a model this small the replaced recompute is also cheap, so the benefit is prefill capacity rather than visible latency; the latency benefit scales with history size and prefill cost, which is what §4’s agentic result realizes.

6 Placement Policy Dominates Transfer Capability

A natural alternative design treats the pull as the primary data path: place requests purely by load and pull the prefix wherever it lands. We tested this on two independent implementations — our own load-first arm on the document-Q&A workload, and an independent

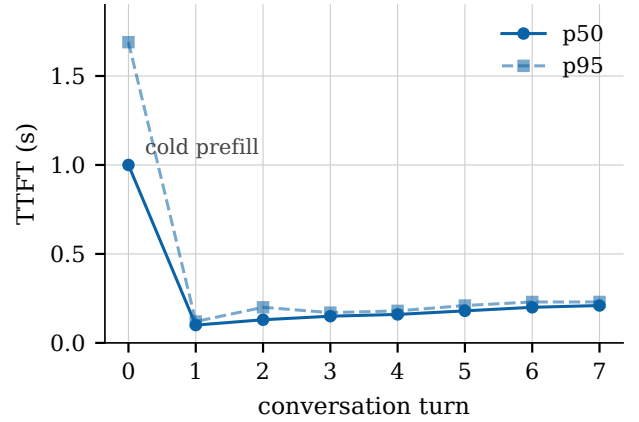


Figure 3: Per-turn TTFT with router-decided prefill←decode pulls (Llama-3.1-8B, 48 conversations × 8 turns). Turn 0 pays the cold prefill; later turns’ history arrives by pull as prompts grow 5K→20K tokens.

wide-expert-parallel deployment at data-parallel degree 16 — and it loses to prefix affinity at every load tested, with the deficit *growing* with concurrency. The failure is structural. Affinity compounds: a pod that serves turn N serves turn $N+1$ from local cache, and its cache mass grows. Load-only placement scatters arrivals, so cache mass never accumulates and the best peer’s coverage per request shrinks as concurrency rises (recomputation share rising from 24.8% to 28.7% across the tested ladder in the independent deployment). Pulled copies also churn the destination tiers, evicting locally accumulated KV (local hit rate falling to 17%), and sustained transfer bandwidth saturates (≈ 4.2 GB/s observed) while affinity’s hits consume none. The winning configuration in every experiment keeps affinity placement and uses the pull as the recovery path for genuine divergence — queue-pressure spills, evictions across idle gaps, cold replicas, migrations — which is also the configuration whose pull volumes are millions of tokens per run rather than terabytes.

7 Limitations

Pulling a *generated* turn requires the follow-up request to reproduce the generated token IDs. Chat-templated APIs achieve this for models whose templates re-render assistant turns verbatim (our Llama-3.1-8B measurement); models that drop reasoning segments on re-render (gpt-oss, Qwen3-Thinking) expose only input context and re-prefilled history for pulling — in the agentic workloads measured here, the bulk of the reuse. Text-append clients (as in common benchmark harnesses) cannot exercise generated-turn reuse for any model, because re-tokenized text diverges from generated IDs. KV block identity is tensor-parallel-dependent in the connector, so peers must run matched TP; and per-peer sessions are established lazily, so the first contact between a pair proceeds as a recompute unless sessions are pre-established.

8 Conclusion

P2P KV-cache sharing converts llm-d’s per-pod prefix caches into a fleet resource under a router-owned, threshold-gated policy. Measured on H200 clusters across three models, it is strictly beneficial as an addition to the P/D reference deployment — 10.3× and 4.8× median TTFT on document-Q&A and agentic workloads with +40% and +33% throughput — provided it is deployed as a recovery path under affinity placement, with the threshold set from a measured per-model crossover.